Course Project Report

RISC-V Processor Implementation

Areesha Kashif, Alishbah Saleem Kath, Sarah Fatima
Instructor: Ms. Maham Tabassum

Processor Summary and Synthesis:

The implemented processor is a single-cycle RV32I design where all stages (fetch, decode, execute, memory, write-back) complete in one clock cycle.

The datapath consists of:

- Program Counter (PC) for instruction sequencing
- Instruction Memory (read-only)
- Register File (32 registers, 2 read and 1 write)
- ALU for arithmetic and logical operations
- Immediate Generator for instruction formats
- Data Memory for load/store operations
- Control Unit (main and ALU control)

Multiplexers control data flow between ALU inputs, write back source, and next PC selection.

Synthesis: The design was written in Verilog and implemented on the Basys 3 FPGA. The standard flow was used:

- Functional Simulation
- Synthesis and Implementation
- Bitstream Generation

A clock divider was used to slow execution for observation.

Task A: Countdown Program Implementation

Task A implements a control-flow program that performs input reading, countdown execution, and continuous looping using jump instructions.

Program Behavior

The program reads an input value from a fixed memory-mapped address and displays it on the seven-segment display. Execution is then transferred to a countdown routine using a `jal` instruction, which stores the return address in `ra`.

Inside the countdown loop, the value is decremented using the ALU and continuously checked using a branch instruction. After completion, control returns to the main program, forming a continuous execution cycle.

Processor Features Used:

- The countdown routine is executed using a `jal` instruction, which stores the return address in `ra` and transfers control to the function.

- Inside the countdown loop, the value is decremented using the ALU and repeatedly checked using branch instructions.
- The stack is used to preserve the return address and temporary registers during function calls, ensuring correct program execution flow.
- Conditional branching is used to control loop continuation and termination, while reset behavior demonstrates external control flow through the FPGA reset signal.
- After completing the countdown, the program returns to the input stage using an unconditional jump, forming a continuous execution cycle.

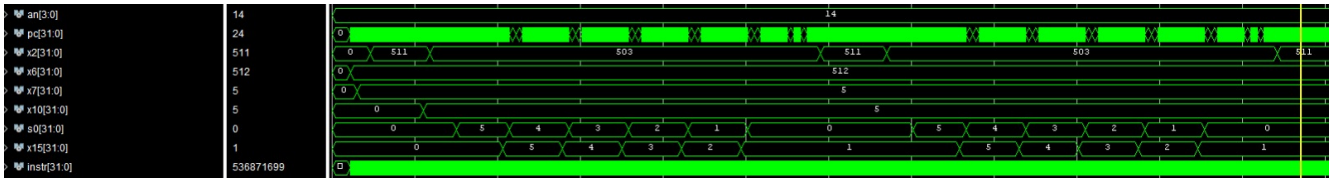

Figure 1: Task A Simulation

Task B: New Instructions

Three new instructions were added: SLTI, SRA, and BLTU.

2.1 SLTI (Set Less Than Immediate)

Operation: rd = (rs1 < imm)

Implemented by extending ALU control for funct3 = 010 under I-type. The ALU performs signed comparison and outputs 1 or 0.

2.2 SRA (Shift Right Arithmetic)

Operation: rd = rs1 >>> rs2

Implemented by distinguishing funct7 bit in ALU control. ALU uses signed shift to preserve sign bits.

2.3 BLTU (Branch if Less Than Unsigned)

Operation: branch if rs1 < rs2 (unsigned)

Implemented by:

- Adding unsigned comparison in ALU
- Extending branch logic to include funct3 = 110

Table: Main Control Unit

Instruction	Type	Opcode	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
SLTI	I	0010011	1	1	0	0	0	0	11
SRA	R	0110011	1	0	0	0	0	0	10
BLTU	B	1100011	0	0	0	0	0	1	01

Table: ALU Control Unit

Instruction	ALUOp	funct3	funct7[5]	ALUControl	Operation
SLTI	11	010	X	0111	Signed Less Than
SRA	10	101	1	1000	Shift Right Arithmetic
BLTU	01	110	X	1001	Unsigned Less Than

Execution / LED Verification

PC (7-seg)	Instruction	LED Pattern [15:4]
12	SLTI	1100 0011 0111
16	SLTI	1100 0011 0111
20	SRA	1000 0010 1000
28	SRA	1000 0010 1000
40	BLTU	0000 0101 1001
48	BLTU taken	0000 0101 1101

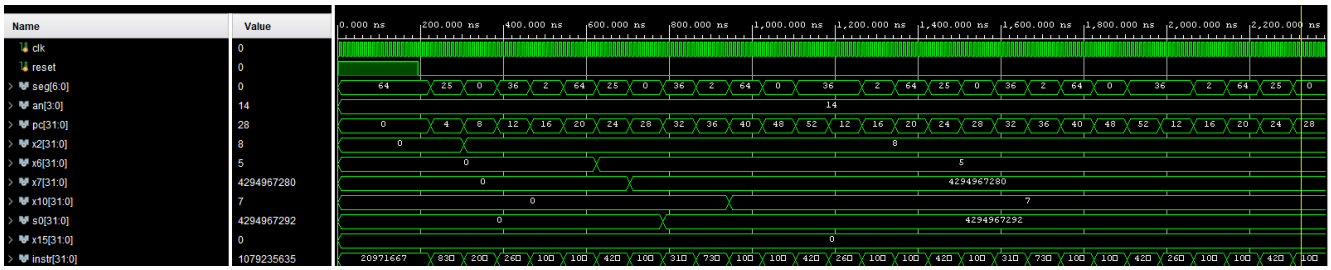

Figure 2: Task B Simulation

Task C: GCD (Greatest Common Divisor) Program Description

A GCD calculator was implemented using the subtraction-based Euclidean algorithm.

Functionality:

- Inputs taken from memory-mapped addresses (0x200, 0x204)
- Start signal from 0x20C
- Output written to 0x208 (7-segment display)

Algorithm:

- If $a > b$, subtract b from a
- If $b > a$, subtract a from b
- Repeat until $a = b$

Implementation Details:

- Uses a subroutine with proper stack handling
- Return address (ra) is saved and restored using stack
- Loop uses conditional branching (including BLTU)

3.1 FSM Representation

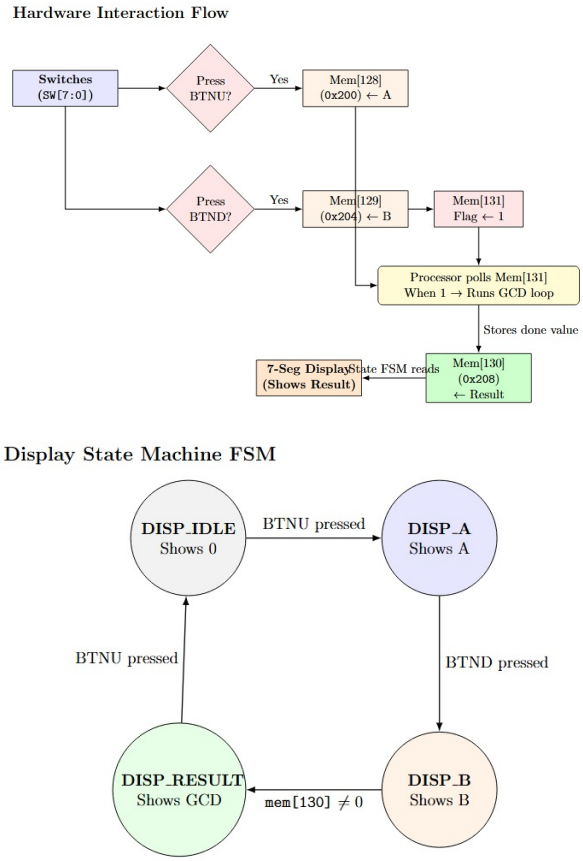

Figure 3: Task C FSM Diagrams

3.2 Simulation

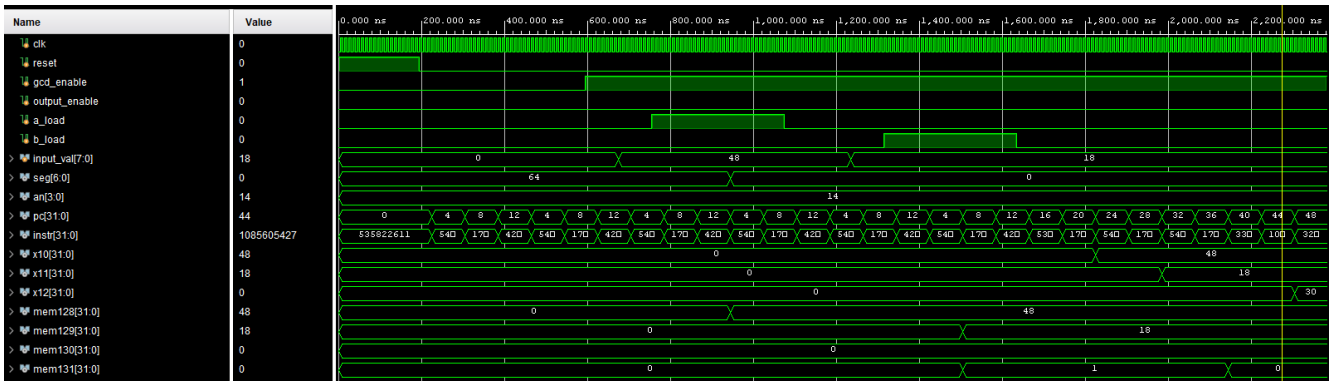

Figure 4: Task C Simulation

Source Code:

GitHub Repository: `https://github.com/Areesha06/CALabs-Git-AreeshaKashif/tree/main/Project`